Laboratory 2 – Diving deeper with Neo4j

Introduction

This laboratory exercise aims at showing databases from a performance and operation point of view. It is again based on Neo4j, and will build on the knowledge acquired during the first laboratory.

For this laboratory, **it is allowed to create teams of two persons**. But you're allowed to work on your own as well.

Essentially, this laboratory consists of loading in neo4j a pretty big graph: the dblp article data. DBLP was an original effort led by University Trier in Germany to create a map of scientific contributions, and this way before scholar.google.com emerged as the leading tool to track papers and citations.

Your goal is to exploit this data to create in Neo4j a graph with more than 10 millions of nodes. But that's not all. We also ask to do so:

1. **without scaling-up too much the environment that loads the data…**
2. **In a decent time…**

Deadlines

We expect you to eventually deliver a report. Please submit it on Moodle by **May 4th, 23h59**. But you also have to deliver a git repository containing the report itself, and also all scripts and code used in the project.

Make sure you grand access to the repo to the teachers. If you go for github, please invite "sebastien-rumley" and "nefu8".

There will also be a Q&A session on April 20th**. There will be no grading at this date, but you are strongly recommended to have something working by then (see below)**.

Dataset

The DBLP dataset is openly available here: https://open.aminer.cn/open/article?id=655db2202ab17a072284bc0c. We will consider the latest dataset (v18) which is available for download here: https://opendata.aminer.cn/dataset/DBLP-Citation-network-V18.zip

But hold on!! Do not download it just now. We have placed an already downloaded, extracted version of the dataset here : http://vmrum.isc.heia-fr.ch/files/DBLP-Citation-network-V18.jsonl

Mind that the compressed file is 5.33 GB big, and once uncompressed, the resulting JSON file is nearly 18GB big. This is a big file to deal with.

At first, you are advised to work with this file : http://vmrum.isc.heia-fr.ch/files/test.jsonl which is a subset.

DO NOT COMMIT the big JSON file into the GIT repository. Anyone doing this will automatically loose the bonus associated with this lab.

Graph Structure

At the end we would like something like that:

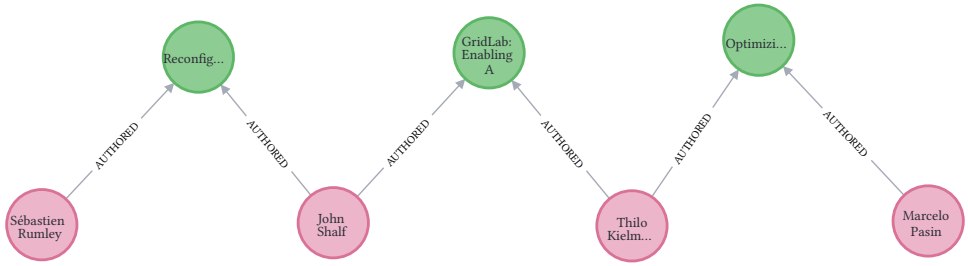

Figure 1: resulting graph.

We want to be able to find thru which co-authors one can relate an author to another, for instance Marcelo and Sébastien (this graph was made with a previous version of the dataset, it might be outdated)

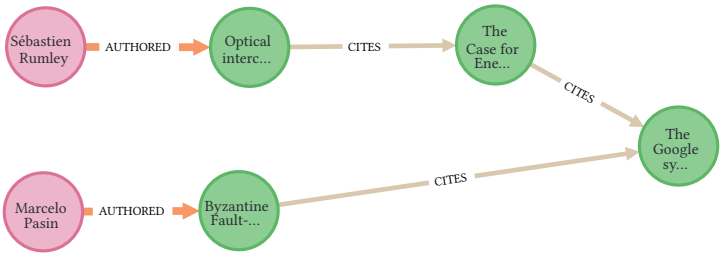

Figure 2: relation graph.

We also want to be able to navigate thru citations. According to a previous version of the dataset, Marcelo and Sébastien had apparently never cited the same paper, but by relaxing the constraint to citations of citations, one can find a "root citation".

So we will build the graph as follows:

- `ARTICLE` nodes
- `AUTHOR` nodes
- `CITES` relationships
- `AUTHORED` relationships
- Each node (`AUTHOR` and `ARTICLE`) with have a key `_id` that corresponds to a key in the JSON file. For instance the `_id` of the first article is `_id : "5390877920f70186a0d2ce7f"`.
- `ARTICLE` nodes will also have a `title` property. For the first instance of article in the JSON, the title is `"title" : "Top-Down Construction of 3-D Mechanical Object Shapes from Engineering Drawings"`
- `AUTHOR` nodes will have a name property. For the second instance of article in the JSON (the first one has no authors), the first author is `"name" : "H YOSHIURA"`
- `AUTHOR` node will be connected to the `ARTICLE` nodes with the `AUTHORED` relationship.
- `ARTICLE` nodes will be related to other `ARTICLE` nodes they cite with the `CITE` relationship.

Initial assignment : local loading

To save you time, an example repository with some of the boilerplate is provided:

https://github.com/sebastienrumley/mse-advDaBa-2022.git

You can either fork it, or download the content and start a new repo from scratch. In this example repo, the Example java code starts the Neo4j driver, reads the 5 first lines of an example JSON file, and creates one dummy node.

You can test it locally by calling `build.sh` which will build an image, and then by calling docker-compose up, which will start a neo4j container, and, once the db is started, will run the example code locally.

Docker limitations

As you can see in the docker-compose file, we want the RAM of the neo4j container to be restricted. In the default configuration, the limit is 3g.

If you want to test without docker-compose, here is more or less the configuration:

```
docker run -it --name advDBlab2  -p7474:7474 -p7687:7687 \
           -v $PWD/neo4j_mount/data:/data \
           -v $PWD/neo4j_mount/conf:/conf \
           -v $PWD/neo4j_mount/logs:/logs \
           --memory="3g"  --env NEO4J_AUTH=neo4j/test  neo4j:4.4.15-community
```

You are advised to use this docker-compose, local approach first to debug and test.

Also note that it is not allowed to copy the dataset inside the container . Data should be streamed from this location : http://vmrum.isc.heia-fr.ch/files/DBLP-Citation-network-V18.jsonl . In fact, the file from which to take the data should be configurable, as shown in the example.

By **April 20th** we'll do a Q&A session, you must have **at least something working locally**, i.e. a version of your solution that is able to load "some" data into Neo4j on your own machine.

Kubernetes

But the real goal of this assignment is to deploy your work as a Kubernetes deployment.

TO OBTAIN A NAMESPACE : Please send a MS Teams message to **Simon Braillard** asking to create a namespace for you, with your name and the name of your team mate. Note that in k8s, strict memory and cpu limits have been set (3G and 2 cpus). This k8s deployment is both to put all groups on an equality basis with the times taken, to strictly enforce memory and cpu limits, and to spare us the task to reproduce all jobs.

At the end of the project, we will want a Kubernetes setup containing:

- one pod running Neo4j, with all the loaded data inside,
- and one pod in charge of inserting the data into the database.

The Neo4j pod must be reachable from May 4th, 23h59 so that we can connect to it and verify that the data has indeed been loaded.

The loading pod must also still be available from May 4th, 23h59, or at least its logs must remain accessible, so that we can inspect how the loading took place.

We will create a namespace for each group. You must therefore clearly indicate which namespace we should inspect.

Your logs must make it possible to see:

- at what time the loading started,
- at what time the loading ended,
- how long it took.
- how many authors, articles and total nodes have been loaded.

This information must be explicit enough so that one can compute the total loading duration from the logs. Please avoid logjams!

You can consult the following documentation to learn how to deploy something on Kubernetes: https://ps4-website.kube-ext.isc.heia-fr.ch/docs/kubernetes/introduction

Final instructions

You are totally free to choose any programming language to load the graph into the DB. The only requirement is that it must be possible to pack it as a docker image, as done in the example.

Mind the `MAX_NODES=10000` variable in the docker-compose descriptor. It is highly recommended to adopt a mechanism to set a limit to the number of nodes that should be loaded. In this way, if needed, one can test your code with a subset of the dblp graph.

We expect pdf report explaining the steps you followed and how your solution works. This report must clearly contain the following information:

- Your group ID (for example `BenammAdvDaBa26`), your names
- The steps you followed and how your solution works
- The namespace in which your jobs and/or deployments can be found (for example `adv-da-ba26-aalamo`)
- The ID of the pod containing Neo4j
- The Neo4j credentials, so that we can verify that the data is correctly stored in the database
- The ID of the pod or pods whose logs prove that the loading actually happened
- The loading time, in seconds, it took to load N + K nodes (N articles, K authors) together with an explanation of how this time can be recovered from the logs
- A link to a clonable git repository containing the code, descriptors, scripts, and all other useful material
- The report should be located at the root of this repository

Good luck!